

Document number: P0831R0
Date: 2017-10-14
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

Keep alias syntax extendable

1. Introduction

This document proposes to reduce the scope of the [P0634: Down with typename!](#) paper, to exclude alias declaration.

Following the current direction of the above paper, the following syntax:

```
using id = some-other-id;
```

will be restricted to type alias declarations, and by doing so, it will prevent its possible further extension: to alias other kind of entities (functions, variables).

2. Motivation and Scope

Features presented in this paper, are used to illustrate the possible extension of the `alias` declaration syntax, that would be blocked by acceptance of [P0634R0](#) paper in its current form. They are **not proposed** as part of this paper.

2.1. Importing function from other namespace

Currently if we want to import an (set of) function `foo` defined in namespace `X` into namespace `Y`, we can use using-declaration syntax:

```
using Y::foo;
```

However, there is no corresponding syntax that would allow to import a function under a different name. As an alternative, we may define a new function `bar` in namespace `X` that would perfect-forward to our target function.

Let our start with the following definition:

```
namespace X {  
    template<typename... Args>  
    decltype(auto) bar(Args&&... args)  
    { return Y::foo(std::forward<Args>(args)...); }  
}
```

For above declaration, there are two major differences between the behaviour of the `X::bar` and `Y::foo` invocations:

- `X::bar` is never `noexcept`, even if corresponding invocation of `Y::foo` would be,
- `X::bar` will lead to hard error, in situation when the invocation of `Y::foo` would lead only to a SFINAE'able error.

To fix both above problem we need to introduce conditional `noexcept` specification and change the return type specification:

```

namespace X {
    template<typename... Args>
    auto bar(Args&&... args)
        noexcept(noexcept(Y::foo(std::forward<Args>(args)...)))
        -> decltype(Y::foo(std::forward<Args>(args)...))
    { return Y::foo(std::forward<Args>(args)...); }
}

```

Still, with the introduction of Concepts Lite, the above declaration may not be equivalent, as function `X::bar` is unconstrained, while the corresponding `Y::foo` function may be, which would lead to selecting a different overload during overload resolution. As consequence, to preserve such behaviour, the user is required to repeat signatures of all of the `Y::foo` overloads.

Finally, even if the user will carefully repeat the signature of the original function, invocation may still may not be equivalent due use of forwarding, that may lead to materialization of temporary and invocation of move constructor. To illustrate, if we have a non-moveable type `NonMovable` and the following declarations:

```

namespace Y {
    void foo(NonMovable);
}

namespace X {
    void bar(NonMovable&& nm) { return Y::foo(std::move(nm)); }
}

```

the invocation `X::bar(NonMovable{})` will be ill-formed, while `Y::foo(NonMovable{})` is fine. This scenario may be considered an edge case, but additional move-construction (that may degenerate to copy) may have a measurable impact on performance.

The author believes that above problems with achieving the simple goal of importing function with different name, shows that the function aliasing feature would be compelling and needed addition to the language — it is worth to remind at this point that none of above issues would occur if we decide to reuse same name and use `using Y::foo;` instead.

2.2. Reusing type alias syntax

As the using-declaration can be used to import any kind of entity from the other namespace, we may expect that the similar type alias syntax could be extended to allow function aliases, leading to the following solution to the problem described in 2.1:

```

namespace X {
    using bar = Y::foo;
}

```

The major concern with this solution, is that it introduces more ambiguity to the language - it is not clear from the syntax what kind of entity is being imported (function, type, template, variable?). To address this, we may follow the namespace alias design, and provide separate syntax for each kind of entity.

On the other hand, if the intent of the user is to make call to `X::bar` equivalent to the `Y::foo`, we may argue, that the syntax should not differ and depend on the fact if the `Y::foo` is implemented as:

- function,
- set of overloaded functions,
- function template,
- set of overloaded function templates,
- set of overloaded functions and function templates,
- global functor variable, mimicking [STL2 customization points design](#).

Otherwise, the selection of appropriate alias syntax and updating it accordingly puts a lot of burden on the user.

The author believes that both above approaches have their own drawbacks and benefits, and blocking one of the solutions at this point would be an mistake, and might impede the future evolution of the language.

3. Acknowledgements

Andrzej Krzemiński offered many useful suggestions and corrections to the proposal.

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal.

4. References

1. Daveed Vandevoorde, "Down with typename!" (P0634R0, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0634r0.pdf>)
2. Eric Niebler, "Suggested Design for Customization Points" (N4381, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4381.html>)